

Christmas Maze

Sistemas Gráficos



Autores

Javier Calvo Gómez
Almudena de la Escalera Esteban

ETSIIT
Escuela Técnica Superior
de Ingenierías Informática
y de Telecomunicación



Índice

1.Plantilla para la concreción del juego.....	3
NOMBRE DEL JUEGO.....	3
Personas que forman el grupo de prácticas.....	3
Descripción.....	3
2. Algoritmos.....	7
2.1. Movimiento por el Laberinto.....	7
2.2. Algoritmo de Colisión.....	8
2.3. Algoritmo de Recogida de PickUps.....	9
2.4. Otros Algoritmos.....	11
2.4.1. AbrirPuerta.....	11
2.4.2. AbrirRegalo.....	12
2.4.3. Movimiento del Reno.....	12
3. Modelos.....	13
4. Técnicas, materiales y luces.....	13

Diseño e implementación de un juego basado en “laberinto”

1.Plantilla para la concreción del juego

Nombre del Juego

The AJ Christmas Maze

Personas que forman el grupo de prácticas

Nombre del/de la Estudiante: Almudena de la Escalera Esteban

Nombre del/de la Estudiante: Javier Calvo Gómez

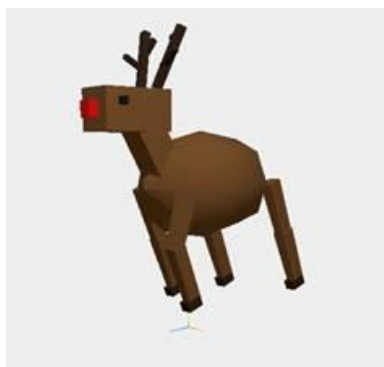
Descripción

Indicar cómo van a ser vuestros pick-ups. Puedes hacer dibujos, incluir fotos, etc.

Para el pick-up articulado, indicar cuáles son sus grados de libertad. NO se pide incluir el modelo jerárquico en este documento, pero sí debe estar en la documentación final de la práctica.

1. Reno

En este caso hemos decidido como pick-up articulado un reno, puesto que nuestra temática es navideña. Tiene en este caso 3 GdL, situados en el cuello dónde moverá la cabeza, y luego para caminar utilizaremos otros dos uno como la rodilla del reno, y como el “hombro” del reno, para que la manera de moverse sea lo más realista posible.



Está es la primera aproximación que hemos realizado en Three.js. Utilizamos como técnicas la de Extrusión y Revolución, consiguiendo así mínimo dos técnicas diferentes en dicho pick-up.

2. Chimenea

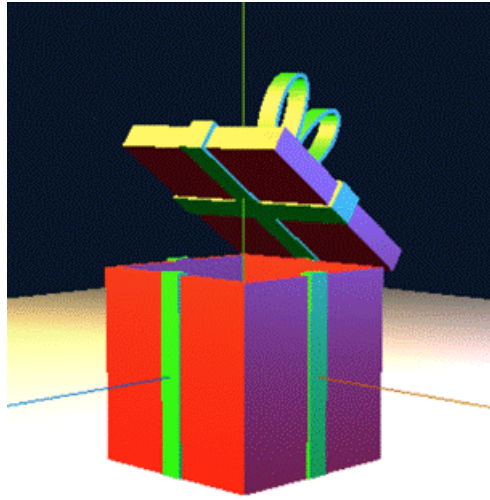
En este caso hemos optado por una Chimenea, como pick-up sólido, aplicándole una textura de ladrillo, y con pensamiento de que en el juego final tenga alguna funcionalidad que nos sea útil. Para la realización de está primera aproximación hemos utilizado Extrusión y CSG, consiguiendo así otras dos técnicas, además añadimos el fuego que es un elemento, que permitirá modificar la iluminación de la estancia.



Primera aproximación lograda. Donde se aprecia el fuego que también chisporrotea.

3. Regalo

En este caso, hemos optado por algo lo más navideño posible, así que hemos realizado un regalo, metiéndole 1 GdL, para poder abrir un regalo, lo que nos puede dar funcionalidades interesantes de cara al juego final. Para su realización usamos extrusión y CSG.



Es la primera aproximación, la apertura se controla usando un GUI.

4. Llave

Ahora vamos a pasar a nuestra llave elegida, como primera aproximación eso sí, en este caso hemos utilizado para su realización, Extrusión, para la estrella del mango y también para la sierra de la llave, implementándolo con holes. Revolución usada para el cuerpo de la llave, y finalmente CSG para la composición general del cuerpo.



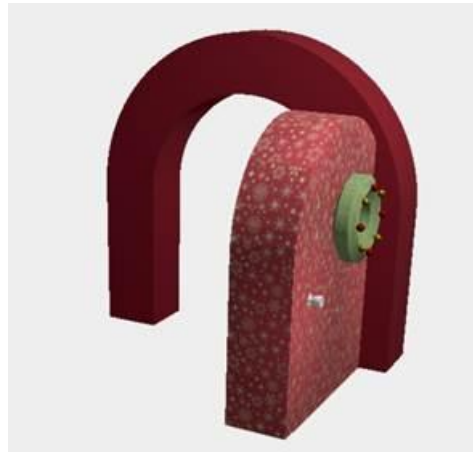
En este caso planteamos el hecho de generar alguna curvatura más en el cuerpo de la llave, para añadirle algo de complejidad a la misma.

Algunos añadidos y la puerta:

5. Puerta

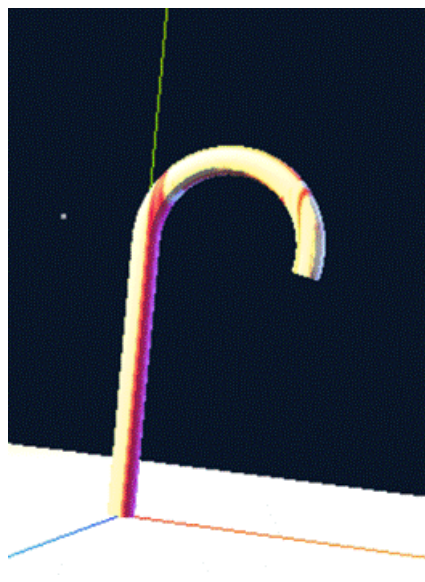
Para la ejecución de la puerta, hemos aprendido, el uso de materiales, así como de objetos importados, tanto externos, como realizados por nosotros como es el caso del rosco decorativo.

También le hemos añadido 1 GdL, para su apertura obviamente. Las técnicas utilizadas son Barrido + CSG. Apreciándose el rosco y la puerta, es aún así nuestra primera aproximación.



6. Bastón de Caramelo

En este caso es el pick-up más simple puesto que no combina dos técnicas, pero supone práctica para la técnica de Barrido, así como un pick-up, meramente decorativo para la temática de navidad.



2. Algoritmos

2.1. Movimiento por el Laberinto

Para el movimiento hemos considerado hacer una clase auxiliar denominado *player*. Esta nos permite tener agrupados variables estáticas para modificar la velocidad tanto de movimiento como de giro, así como el radio que luego tendremos en cuenta para el raycast, con objetos y muros.

Para la rotación se han de tener en cuenta las dos formas posibles de movimiento, que son en el caso de que se muevan en el *modoRaton*, es decir que sea controlado por el movimiento que se captura del ratón, y en el caso de que sean movimientos que se realicen mediante botones de giro en este caso 'a' y 'd'. Se tendrá en cuenta en el *update* de player.

```
if(!locked){
    if( teclas['a'] || teclas['ArrowLeft']) this.angulo += Player.VEL_GIRO;
    if( teclas['d'] || teclas['ArrowRight']) this.angulo -= Player.VEL_GIRO;
    this.rotation.y = this.angulo;
}
```

Imagen 2.1 : Como tiene en cuenta las teclas.

Como se puede observar en la Imagen 2.1, si no está bloqueado para el uso del ratón. Si bien es cierto que en el caso del movimiento hacia delante y hacia detrás, que se realiza con las teclas 'w' y 's' respectivamente.

```
//Avanzar/Retroceder
let dir = 0; //Lógica 1: Adelante y -1: Atrás
if( teclas['w'] || teclas['ArrowUp']) dir = 1;
if( teclas['s'] || teclas['ArrowDown']) dir = -1;
```

Imagen 2.2: Muestra como es el movimiento normal.

Una vez comprobado si se ha pulsado o no la tecla, el movimiento solo es funcional siempre y cuando *dir* sea distinto de 0, pero para el movimiento también es necesario saber si te encuentras en *modoRaton*. En caso de que eso sea cierto se debe obtener la dirección a la que mira la cámara en el momento que el botón sea pulsado, una vez obtenido la dirección y normalizada la misma, se aplica *moveForward(Player.VELOCIDAD)* con el fin de aplicar la velocidad que se requiere. En el caso de que el botón pulsado sea el de marcha atrás nos ayudamos de la función dada *negate* que nos permite invertir el vector para que el movimiento se aplique en esa dirección.

En caso de que no nos encontremos en *modoRaton* la dirección se calcula de forma más sencilla, con un vector *direccion* que para sus posiciones x y z tendrán en cuenta el ángulo acumulado gracias a *this.angulo*. También, multiplicado por el valor de *dir* que es lo que permite comprobar si es para adelante o para atrás. Y se le aplica el *Player.VELOCIDAD*.

```

if (pointerLocker && locked) {
    const aQueMiro = new THREE.Vector3();
    pointerLocker.getDirection(aQueMiro);
    aQueMiro.y = 0;
    aQueMiro.normalize();
    if (dir === -1) aQueMiro.negate();
} else {
    const direccion = new THREE.Vector3(
        Math.sin(this.angulo) * dir * -1,
        0,
        Math.cos(this.angulo) * dir * -1
    ).normalize();
}

```

Imagen 2.3: A la izquierda en *modoRatón*, y a la derecha en modo normal

Sin embargo, si bien ahora se consigue un movimiento fluido es necesario tener en cuenta que entre medias es necesario tener en cuenta los posibles choques con los objetos y muros.

2.2. Algoritmo de Colisión

Vamos a observar como se ha implementado la detección de picks ups y de muros. Para el caso de los muros se ha determinado realizarlo mediante raycaster, que consiste en lanzar un rayo desde un punto en una dirección y devolviendo todo lo que interseca.

Se genera un variable estática que establezca el radio para inicializar el *far* del *raycaster* con esa distancia. Que es el radio del jugador para identificar el muro. Una vez creado el raycaster hay que volver a diferenciar entre los casos de que esté activo el *modoRaton* o no. Aunque el proceso es prácticamente igual sólo variarán los parámetros que se le pasen.

```

this.raycaster.set(origen, direccion);
const chocaMuro = this.raycaster.intersectObjects(muros || [], true).length > 0;

```

Imagen 2.4: Como generar el raycast

En la Imagen 2.4 se observa el caso de que no esté activo el *modoRaton*, siendo origen un vector que captura la posición actual del jugador y elevado un metro en y puesto que si no intersecan con el suelo, y si estuviésemos en *modoRaton*, se le pasaría en vez de *direccion*, *aQueMiro*. Finalmente requerimos de un booleano para ver si en el radio de *Player.RADIO* hay o no un muro.

Sin embargo, para los pickups usar raycaster puede ser más limitado en ciertos aspectos, puesto que también hay que tener en cuenta si estamos encima o no. Es decir que podríamos estar mirando a otro lado y estar encima pues el rayo no lo detectaría. Es por ello que se ha decidido calcular la distancia horizontal entre el jugador y los pickups. Si dicha distancia es menor de 0.35m entonces el pickup bloquea el paso.


```

const pickupBloqueaPaso = (origen, direccion) => {
  ...if (!pickups || pickups.length === 0) return false;

  ...const siguientePos = origen.clone().add(direccion.clone().multiplyScalar(Player.VELOCIDAD));

  ...for (const pickup of pickups) {
  ...  ...if (!pickup || !pickup.getWorldPosition) continue;

  ...    ...const posPickup = new THREE.Vector3();
  ...    ...pickup.getWorldPosition(posPickup);

  ...    ...const radioPickup = 0.35;
  ...    ...const distanciaXZ = Math.hypot(
  ...    ...  posPickup.x - siguientePos.x,
  ...    ...  posPickup.z - siguientePos.z
  ...    ...);

  ...    ...if (distanciaXZ < radioPickup) {
  ...    ...  return true;
  ...    ...}
  ...  }

  ...return false;
};

```

Imagen 2.5: Función para aplicar la distancia.

2.3. Algoritmo de Recogida de PickUps

En este caso, se ha decidido otra vez dos formatos teniendo en cuenta si está o no en *modoRaton*. Aunque de todas formas se hará dándole 'e' o clicando. Lo primero es saber que todo se hace en el *MyScene*, usaremos un array para almacenar todos los pickups que existen dentro del laberinto.

Para la ejecución se realiza a través de una función llamada *recogerPickup*. Al ejecutarse, lo primero que hace es comprobar si existe o no el jugador, y si el array está o no vacío. A continuación, recorre dicho array de atrás hacia adelante. Orden importante pues eliminamos los pickups del array mediante *splice*.

Para cada pickup se obtiene su posición en el mundo *getWorldPosition* y se calcula la distancia hacia el jugador mediante *distanceTo*. Estableciendo como umbral 1,6 metros, para empezar a considerar el pickup alcanzable. Una vez dentro, para poder actualizar correctamente los UI relativos a cada pickup se ha de distinguir qué tipo se ha recogido. Así como tener en cuenta que hay variables o pickups que requieren de funciones auxiliares, como el regalo que se realiza la animación de abrirse, o la chimenea para establecer la luz en todo el mapa.

```

recogerPickup() {
    if (!this.jugador || !this.pickups) return;
    const distanciaRecogida = 1.6;
    let recogidoAlgo = false;

    for (let i = this.pickups.length - 1; i >= 0; i--) {
        if (recogidoAlgo) break;
        const pickup = this.pickups[i];
        if (!pickup || !pickup.position) continue;

        const posPickup = new THREE.Vector3();
        pickup.getWorldPosition(posPickup);
        const distancia = this.jugador.position.distanceTo(posPickup);

        if (distancia < distanciaRecogida) {
            let tipoEncontrado = null;
            // Identificar tipo y actualizar estado
            if (pickup instanceof coponieve) {
                this.tengoLlave = true;
                tipoEncontrado = '🔑 Llave';
                console.log('¡Has recogido la llave!');
            } else if (pickup instanceof Regalo) {
                if (!pickup.abierto) {
                    this.abrirRegalo(pickup);
                    recogidoAlgo = true;
                    break;
                }
                tipoEncontrado = '🎁 Regalo';
                console.log('¡Has recogido un regalo!');
            } else if (pickup instanceof Chimenea) {
                tipoEncontrado = '🔥 Chimenea';
                console.log('¡Has encontrado la chimenea!');
            } else if (pickup instanceof Campana) {
                tipoEncontrado = '🔔 Campana';
                console.log('¡Has recogido la campana!');
            }
            // Reproducir sonido de campana
            // Reproducir sonido de campana
            const campanaSound = new Audio('../sounds/campana.mp3');
            campanaSound.play().catch(err => console.warn('Error al reproducir sonido:', err));
        } else if (pickup instanceof Reno) {
            tipoEncontrado = '🐐 Reno';
            console.log('¡Has recogido al reno!');
        }

        if (tipoEncontrado && this.pickupStatus) {
            const estado = this.pickupStatus.find(item => item.objeto === pickup);
            if (estado && !estado.collected) {
                estado.collected = true;
                this.updatePickupUI();
            }
        }

        // Si recogemos la chimenea, habilitar las luces del laberinto
        if (pickup instanceof Chimenea) {
            this.tengoLlave = this.tengoLlave || false; // no afecta pero mantiene semántica
            this.lightsEnabled = true;
            this.enablelights();
            console.log('🔥 Chimenea recogida: iluminando el laberinto');
        }

        // Eliminar físicamente el objeto
        this.laberinto.remove(pickup);
        if (pickup instanceof Reno) {
            const idx = this.laberinto.renosPatrulla.indexOf(pickup);
            if (idx !== -1) this.laberinto.renosPatrulla.splice(idx, 1);
        }
        this.pickups.splice(i, 1);

        pickup.traverse((hijo) => {
            if (hijo.isMesh) {
                const idx = this.obstaculos.indexOf(hijo);
                if (idx !== -1) this.obstaculos.splice(idx, 1);
            }
        });
        recogidoAlgo = true;
    }
}

```

Imagen 2.6: Método *recogerPickup*

2.4. Otros Algoritmos

2.4.1. AbrirPuerta

La puerta sólo puede ser abierta, si has conseguido recoger todos los pickups del juego, y si el jugador se encuentra a una distancia determinada, para que no se abra desde cualquier punto del mapa. Al igual que la mayoría de los algoritmos vistos, se puede abrir con la 'q' pero como bien decía la práctica lo más importante es haciendo click sobre el pomo. Para detectar el click con el pomo se usa un *raycaster* recorriendo todos los padres del objeto intersectado, para eu solo interactúe en el caso de que éste sea el pomo.

```
abrirPuerta() {
  if (!this.jugador || !this.laberinto) return;
  // Requerir que se hayan recogido todos los pickups (si los hay)
  const total = this.pickupStatus ? this.pickupStatus.length : 0;
  const collected = this.pickupStatus ? this.pickupStatus.filter(i => i.collected).length : 0;
  if (total > 0 && collected < total) {
    console.log(' No puedes abrir la puerta: recoge todos los objetos primero. ');
    return;
  }
  const distanciaApertura = 10.5;
  const posSalida = this.laberinto.getPosFinal();
  const distanciaJugadorSalida = this.jugador.position.distanceTo(posSalida);
  if (distanciaJugadorSalida < distanciaApertura) {
    this.puerta.abrirPuerta();
    this.puertaAbierta = true;
    console.log(' ■ Puerta abierta! ');
  }
}
```

Imagen 2.7: Método *abrirPuerta*

```
isKnobClick(event) {
  if (!this.puerta || !this.camaraJugador) return false;
  this.mouse.x = (event.clientX / window.innerWidth) * 2 - 1;
  this.mouse.y = -(event.clientY / window.innerHeight) * 2 + 1;
  this.raycaster.setFromCamera(this.mouse, this.camaraJugador);
  const intersects = this.raycaster.intersectObjects([this.puerta], true);

  for (const inter of intersects) {
    let obj = inter.object;
    while (obj) {
      if (obj.userData && obj.userData.isKnob) return true;
      obj = obj.parent;
    }
  }

  return false;
}
```

Imagen 2.8: Método para comprobar si es el pomo.

2.4.2. AbrirRegalo

Al tratar de coger de primeras el regalo, no interactúa como el resto de pickups, es decir, no lo coges de primeras sino que realiza una apertura puesto que es el que contiene la llave que necesitas. Solo en ese momento la llave pasa a ser un pickup dentro del array de pickups.

```
_abrirRegalo(regalo) {
    const luzRegalo = new THREE.PointLight(0xffd700, 0, 3);
    const posRegalo = new THREE.Vector3();
    regalo.getWorldPosition(posRegalo);
    luzRegalo.position.copy(posRegalo);
    luzRegalo.position.y += 0.15;
    this.add(luzRegalo);
    regalo._luzRegalo = luzRegalo;

    regalo.abrirAnimado(() => {
        const llave = new coponieve();
        llave.scale.set(3, 3, 3);

        const pos = new THREE.Vector3();
        regalo.getWorldPosition(pos);

        llave.position.set(
            pos.x - this.laberinto.position.x,
            0.05,
            pos.z - this.laberinto.position.z
        );

        const luzHielo = new THREE.PointLight(0xaaddff, 3, 3);
        luzHielo.position.set(0, 0.3, 0);
        llave.add(luzHielo);

        this.laberinto.add(llave);
        this.pickups.push(llave);

        // Animación de subida
        llave._subiendo = true;
        llave._alturaFinal = 0.35;
        llave._velocidadSubida = 0.008;

        this.pickupStatus.push({ nombre: '🔑 Llave (regalo)', collected: false, objeto: llave });
        this.updatePickupUI();
        luzRegalo.intensity = 0.3;
        console.log('📦 Regalo abierto: la llave emerge');
    });
}
```

Imagen 2.9: Método *abrirPuerta*

2.4.3. Movimiento del Reno

El reno patrulla por el laberinto para aprovechar que es articulado y realizar su animación de caminar, usando un camino predefinido de celdas, hasta llegar al final y repetir el camino y teniendo en cuenta si la celda es o no transitable.

```
_actualizarPosicionReno(reno){
    if(!this.laberintoMatriz) return;

    const objetivoX = reno._colObj * this.anchobloque;
    const objetivoZ = reno._filaObj * this.anchobloque;

    const dX = objetivoX - reno.position.x;
    const dZ = objetivoZ - reno.position.z;
    const distancia = Math.sqrt(dX * dX + dZ * dZ);

    if(distancia < 0.05){
        reno.position.set(objetivoX, 0, objetivoZ);
        reno._filaAct = reno._filaObj;
        reno._colAct = reno._colObj;
        this._elegirNuevoObjetivo(reno);
    } else {
        const velocidad = 0.02;
        reno.position.x += (dX / distancia) * velocidad;
        reno.position.z += (dZ / distancia) * velocidad;
        reno.renoGroup.rotation.y = Math.atan2(dX, dZ);
    }
}
```

Imagen 2.10: Método *_actualizarPosicionReno*

```

_elegirNuevoObjetivo(reno){
  const direcciones = [
    { fila: reno._filaAct - 1, col: reno._colAct },
    { fila: reno._filaAct + 1, col: reno._colAct },
    { fila: reno._filaAct, col: reno._colAct - 1 },
    { fila: reno._filaAct, col: reno._colAct + 1 }
  ];

  const transitables = direcciones.filter(d => this.esCeldaTransitable(d.fila, d.col));
  if (transitables.length === 0) return;

  // Evitar volver a la celda anterior si hay otras opciones
  const sinRetroceso = transitables.filter(
    d => !(d.fila === reno._filaAnterior && d.col === reno._colAnterior)
  );
  const opciones = sinRetroceso.length > 0 ? sinRetroceso : transitables;

  const elegido = opciones[Math.floor(Math.random() * opciones.length)];
  reno._filaAnterior = reno._filaAct;
  reno._colAnterior = reno._colAct;
  reno._filaObj = elegido.fila;
  reno._colObj = elegido.col;
}

```

Imagen 2.11: Método para elegir el nuevo objetivo.

3. Modelos

El pomo lo hemos sacado de:

<https://free3d.com/3d-model/tulip-entry-door-knob-v3--542052.html>

Los sonidos los hemos sacado de:

<https://pixabay.com/es/sound-effects/search>

4. Técnicas, materiales y luces

A pesar de indicar en el documento inicial como queríamos plantear cada pick up, vamos a recalcar como han sido hechos finalmente cada uno de ellos. Incluyendo texturas usadas para cada una de ellas.

- **Chimenea:** La Chimenea se ha optado por usar una combinación entre Extrusión y CSG. Aprovechando una imagen exportada, hemos aplicado una textura similar a la del ladrillo. La textura se puede observar en la imagen del documento inicial. Mientras que para el fuego se juega con la combinación de *emissive* y *roughness*.
- **Regalo:** Para el Regalo se ha optado por la misma combinación que el pickup inicial, mezclando el uso de Extrusión y CSG. Para la textura, hemos optado por

MeshStandardMaterial, modificando los parámetros de *color* y *roughness*, además de utilizar en el caso del contorno de la caja *side* : *THREE.DoubleSide*.

- **Llave:** Para la Llave se ha optado por un conjunto de tres técnicas tales como Extrusión, CSG y Barrido. Mientras que para la textura también, se ha usado un *MeshStandardMaterial* en el que aparte de los anteriores se ha modificado *metalness* y *emissive*.
- **Bastón de Caramelo:** Se ha usado la técnica de barrido, más una textura adicional, para hacerlo lo más similar posible al material real.
- **Rosco + Puerta:** La puerta está formada por diferentes partes creadas cada una de distinta manera, mezclando el uso de Barrido y CSG, junto con la aplicación de un objeto cargado como es el pomo de la puerta. Para la puerta se ha aplicado texturas cargadas a las que se les ha aplicado un bump, para permitir observar un relieve.
- **Muro:** Para el muro no se ha usado una técnica concreta para crearlo, pero se han añadido texturas para mejorarlas mediante bump. Además como adicional, recalcar que al **suelo** también se le ha aplicado una textura realizada mediante bump.
- **Reno:** Para nuestro objeto articulado se ha aplicado Barrido junto con revolución, en el que la textura solo viene modificada por el color.
- **Campana:** Por último la campana está hecha mediante Revolución. Y dónde nos hemos centrado en la combinación de valores entre *metalness* y *roughness*.

A parte de lo comentado, hay pickups que tienen relación directa con las luces creadas para el juego, ejemplo de ello son:

- **Campana:** Cuenta con unas luces rectangulares que apuntan directamente hacia ella. Para ello se debe obtener la posición de la campana en el laberinto, para poder crearla alrededor de ella. En el juego se ven como se aprecia en la Imagen 4.1.

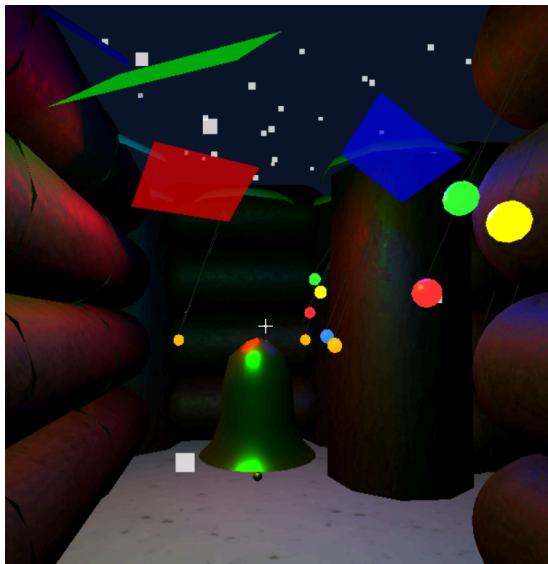


Imagen 4.1: Luces de la campana.

- **Chimenea:** Tiene dos aplicaciones sobre la luz.
 - La primera de ellas consiste en un **SpotLight**. El laberinto de primeras está a oscuras por una de las mecánicas del juego, pero para mayor facilidad se le añade un spotlight a la chimenea para que al menos esté sea fácil de identificar. Sin embargo, este spotlight puede ser quitado desde los controles del juego.

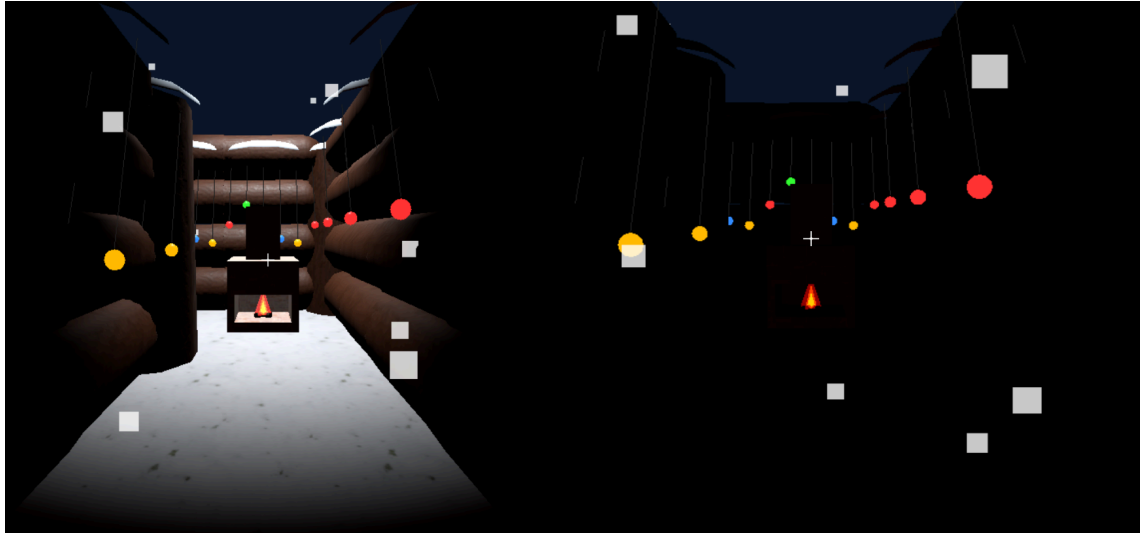


Imagen 3.2: Muestra el contraste entre si hay o no SpotLight.

- La segunda opción que posee la chimenea, está relacionado con el tema de la oscuridad del laberinto, pues hasta que está no sea seleccionada no volverá la luz al laberinto.
- **Regalo:** El regalo como ya vimos tiene la cualidad de que puede abrirse, y hacer aparecer la llave. Ambos objetos producen una luz, uno al abrirse y otro al aparecer, para que se vea mejor dentro del laberinto, como se muestra en la Imagen 3.3.



Imagen 3.3: Muestra la luz que se genera al abrir el regalo.

6.2. Regalo

